Lab 6: Build a Web Server in Public Clouds

Public clouds enable on-demand cloud resources to be provided to users via the Internet. There are various public cloud services such as infrastructure as a service (IaaS), platform as a service (PaaS), or software as a service (SaaS), etc., and users are charged according to the usage.

This lab session will allow students to explore public cloud services and build a simple web server based on IaaS. Students can learn how to launch a compute instance in public clouds, develop their own cloud-based web service, and monitor the performance.

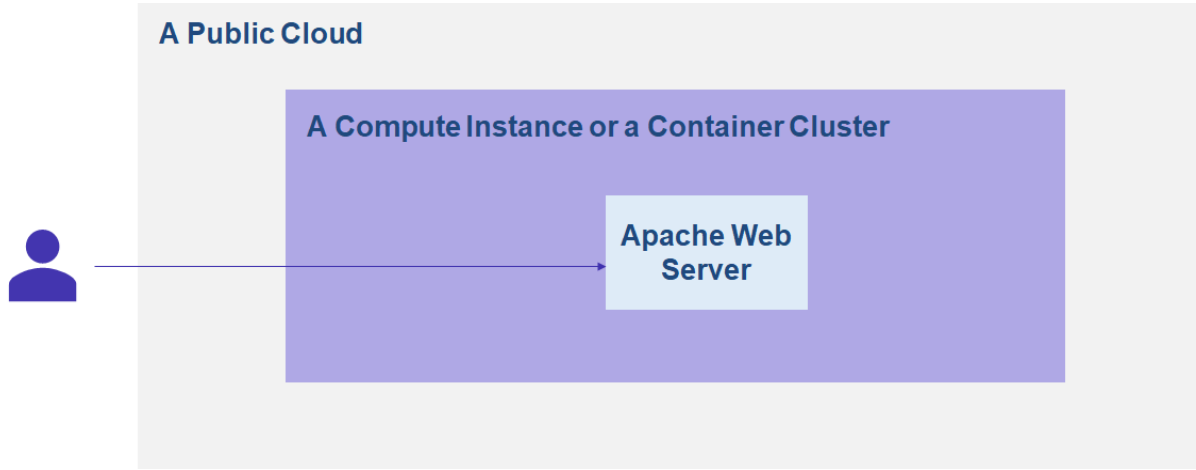
Topics covered

This lab will cover three parts:

- Part 1, to launch an Amazon **Elastic Compute Cloud (EC2)** instance and build a simple web server using Apache web server and a simple Python-programmed microservice using Docker container with Flask;
- Part 2 (optional), to launch a **Google Compute** Engine instance and build a simple web server; and
- Part 3 (optional), to launch an **Azure Virtual Machine** instance and build a simple web server.

Students need to complete ***Part 1*** to build and run a web server in Amazon Web Services (AWS) cloud while Part 2 and Part 3 are optional. A high-level system architecture is shown below.

Tasks 1-3



A high-level view of the lab experimental environment

Task 4

Develop a Python-programmed microservice on a Docker container @ EC2.

Learning Objectives

- Launching a web server using an Amazon EC2, a Google Compute Engine, or an Azure Virtual Machine instance.
- Create a Python-programmed microservice using a Docker container
- Configure the web server capacity
- Monitoring the performance of the web server
- Terminating the compute instances

Duration

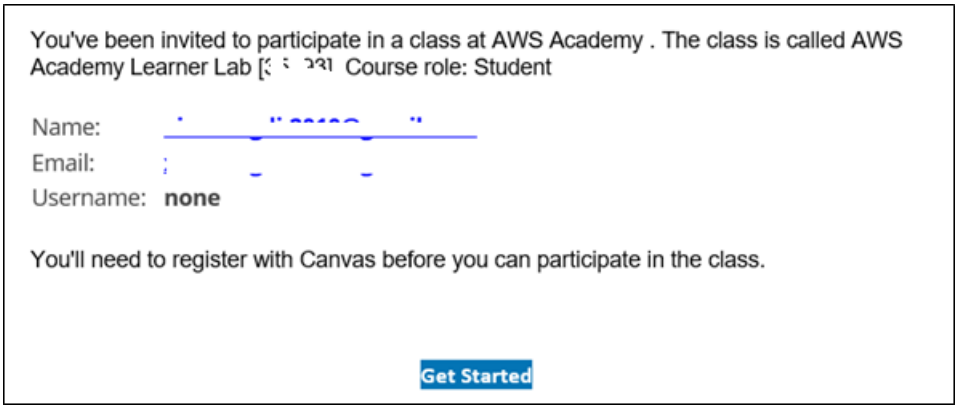
This lab takes approximately **120 minutes** to complete.

Part 1: Build a Web Server in AWS Public Cloud

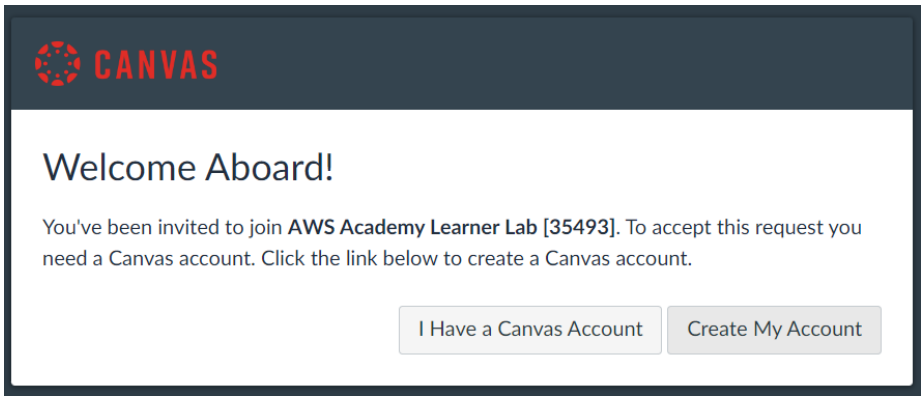
An Amazon EC2 instance will be launched to run a simple web server.

Task 0: Log in the AWSAcademy web portal

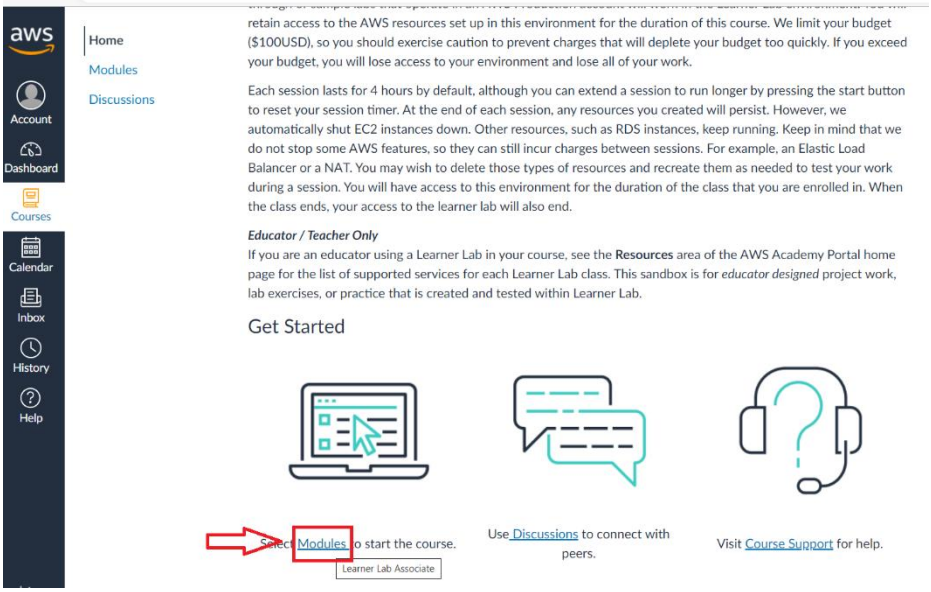
Accept the invitation to Academy Learner Lab course by clicking Get Started.



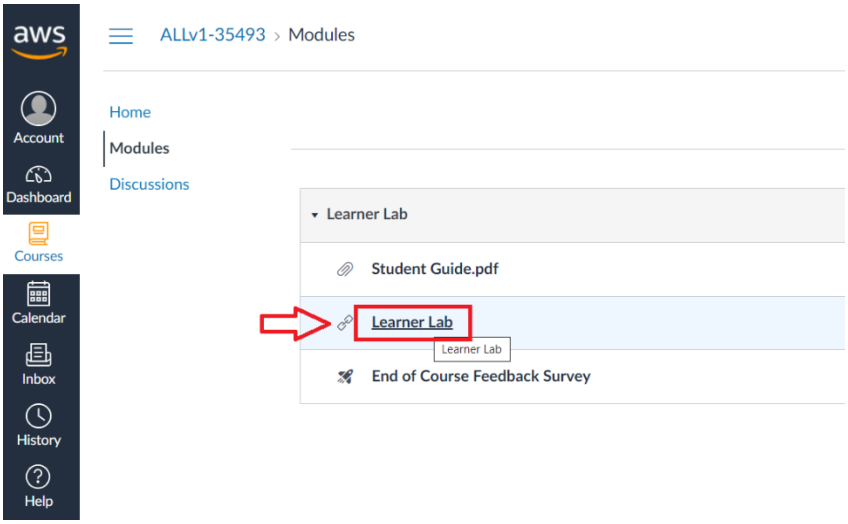
Click Create My Account to create an AWSAcademy Canvas account.



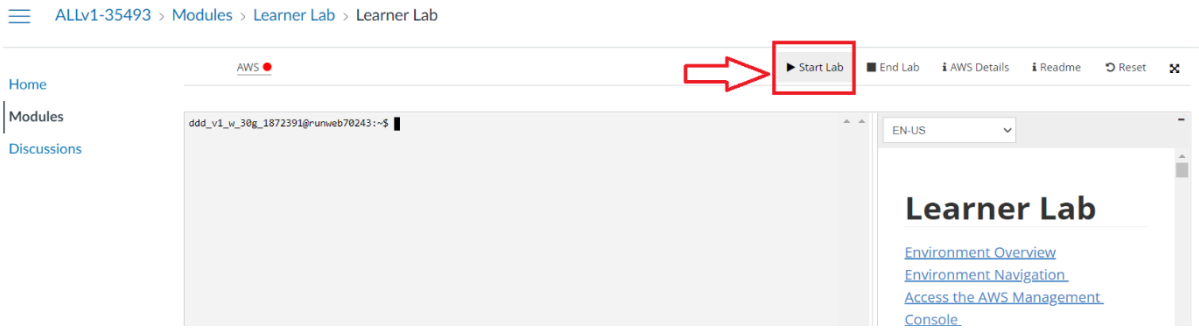
Click Modules to start the course



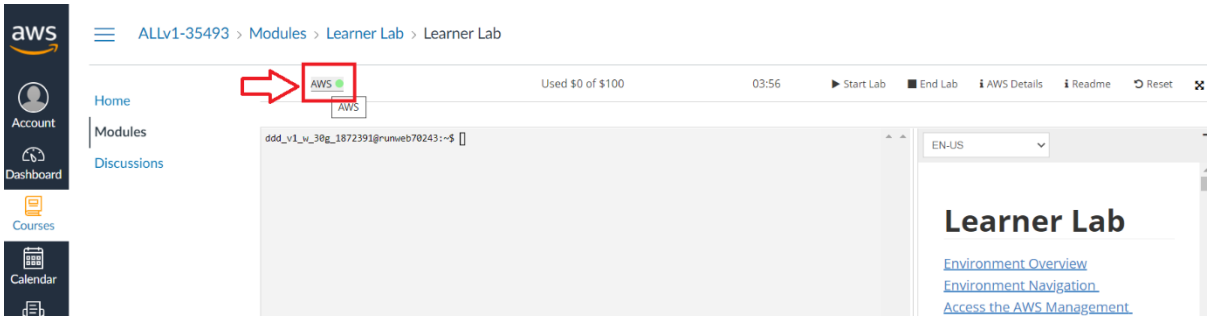
Click Learner Lab and Click I Agree to agree the terms and conditions.



Click Start Lab

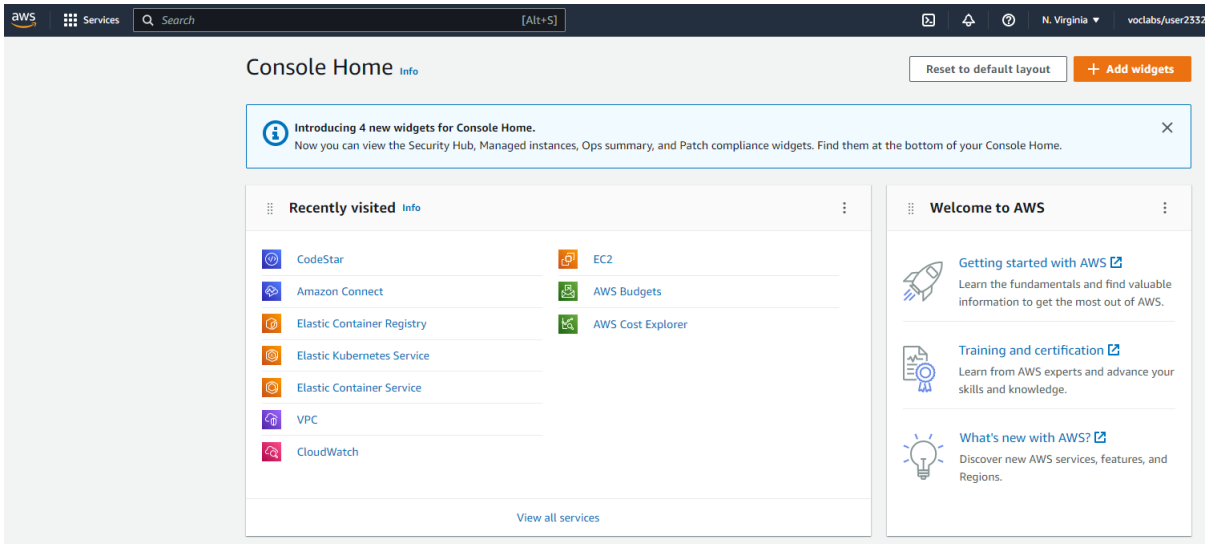


Once the bubble beside the AWS becomes green, click AWS to go to the web console of Amazon Web Services (AWS) cloud.



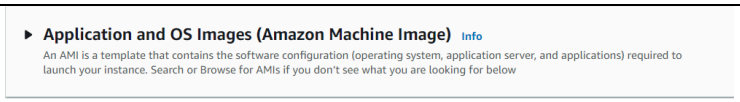
There are USD$100 AWS credits for cloud computing lab experiments @AWS cloud. It is an invitation based and for CSD3156 students in Tri 2 2023 only.

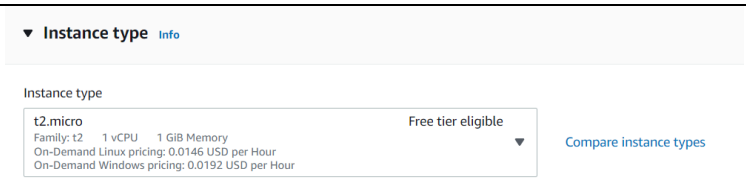
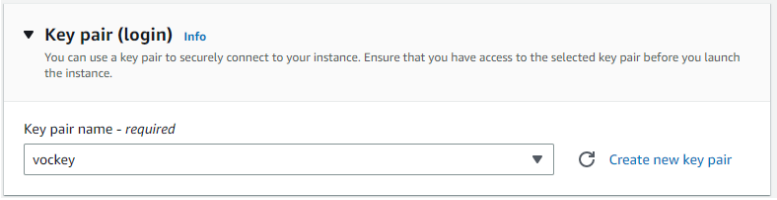
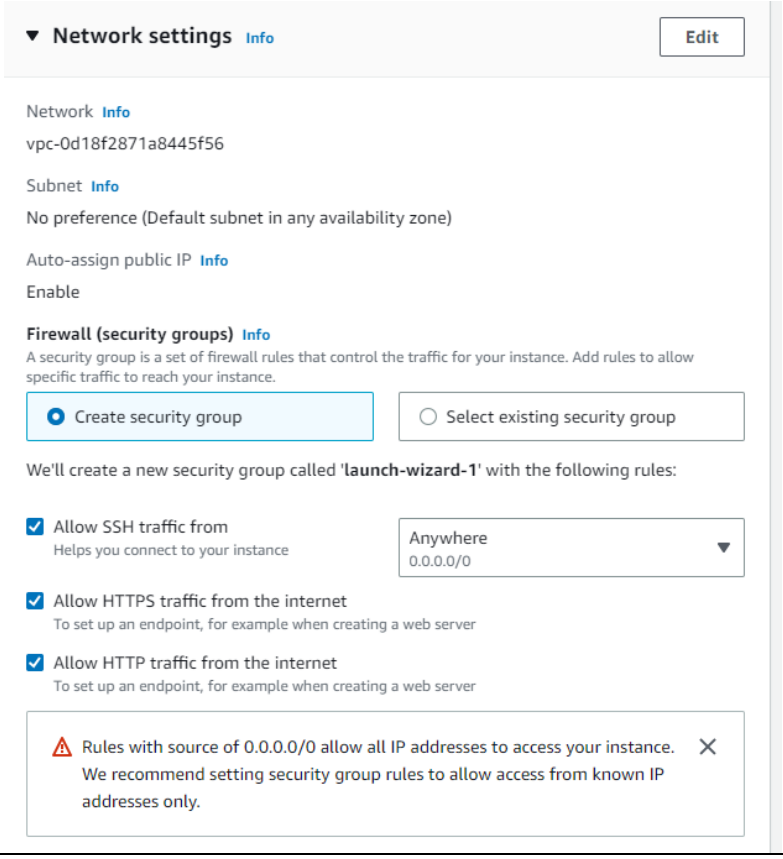
You can select different cloud services from AWS console now.

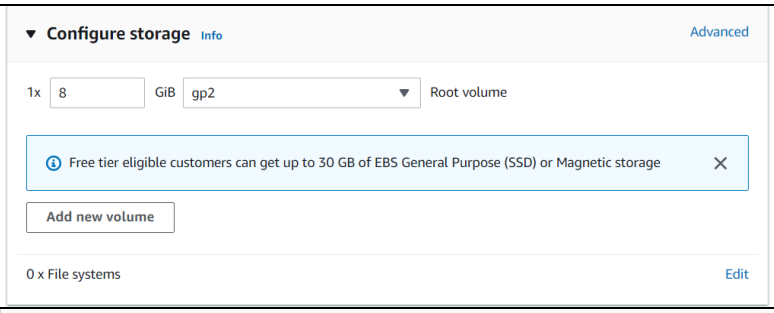
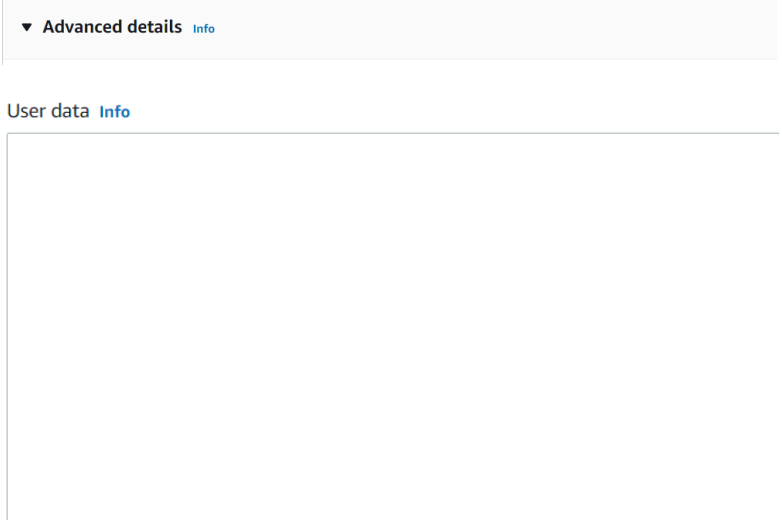


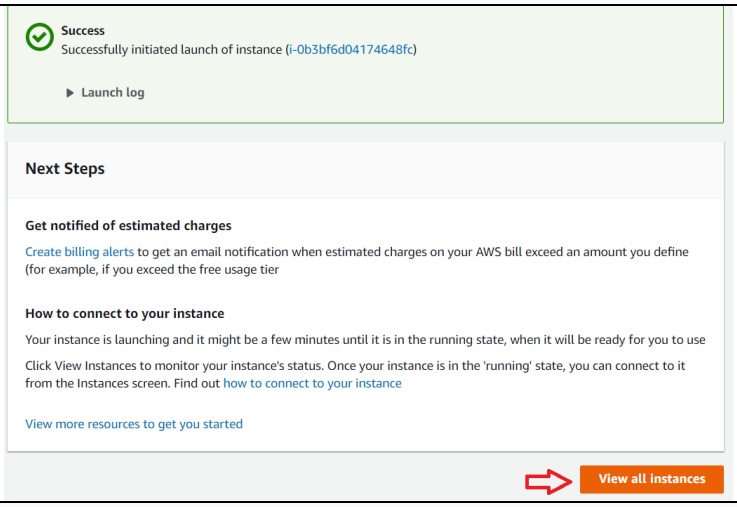
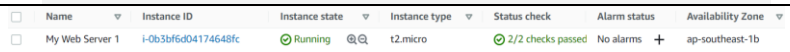
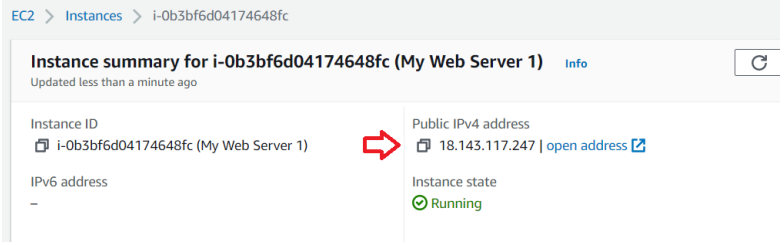
Task 1: Launch a web server using an Amazon EC2 instance

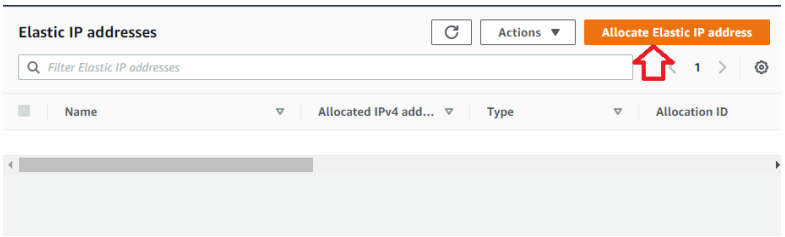
Go to **AWS Management Console**, choose **Compute** in the search box to the right of **Services,** select **EC2,** and choose **Launch instances.**

SN	Steps	Instructions
1)	Name and Tag: Name the EC2 instance as "**My Web Server 1**"	
2)	Choose the default **AMI**	

3)	Choose the default instance type **t2.micro**	
4)	Use the default key pair: **vockey**	
5)	**Network settings**	Create security group and select SSH, HTTPS, and HTTP traffic. 

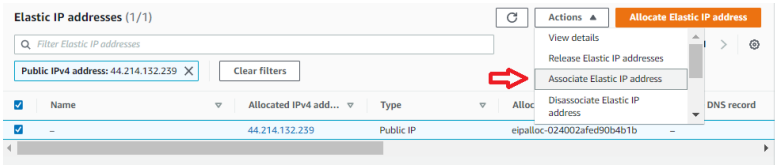
6)	**Configure storage**	
7)	Expand **Advanced Details** to configure the user data	 Copy the following lines into the User data info. #!/bin/bash # Update package and start Apache web server sudo yum update -y sudo yum install httpd -y sudo systemctl start httpd sudo systemctl enable httpd # Create a simple index.html file echo '<html><h1>Hello From Web Server 1!</h1></html>' > a.html sudo cp a.html /var/www/html/index.html rm a.html # Set appropriate permissions sudo chown apache:apache /var/www/html/index.html sudo chmod 644 /var/www/html/index.html # Display server status sudo systemctl status httpd The *user data* will perform automated installation and configuration tasks after the instance starts. The script will:

		○ Install an Apache web server (httpd) ○ Configure the web server to automatically start on boot ○ Run the Web server once it has finished installing ○ Create a simple web page
8)	Launch instance	
9)	View all instances	
10)	Check the status	 The instance has been successfully initialised and checked once Instance state shows Running and Status check becomes 2/2 checks passed.
11)	Get the public IP address	 ***Troubleshooting 1***: If there is no Public IP address allocated to the instance, go to Network & Security in the left pane, click Elastic IPs, and choose **Allocate Elastic IP address**.

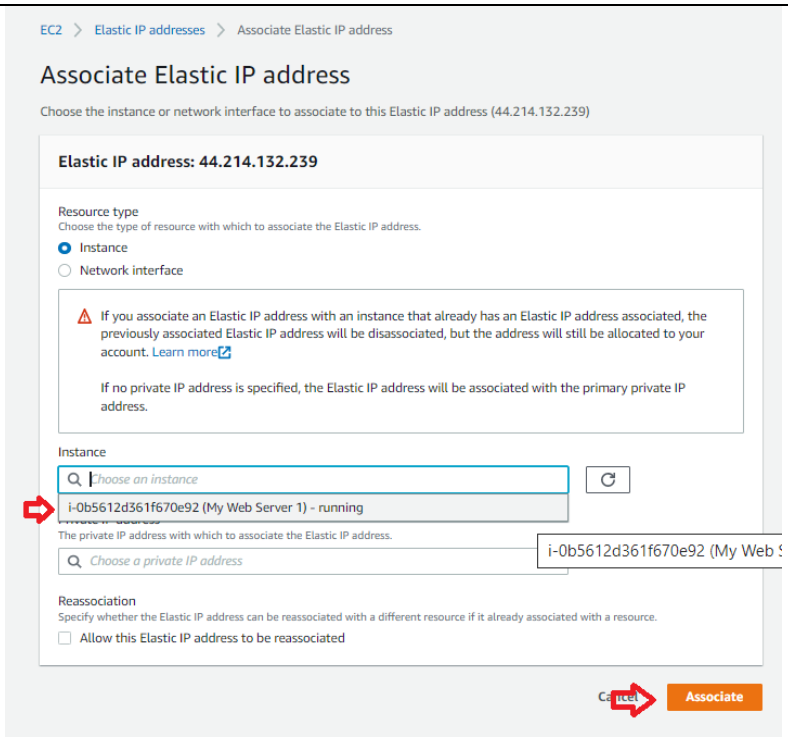


Click **Allocate** to generate an Elastic IP address.

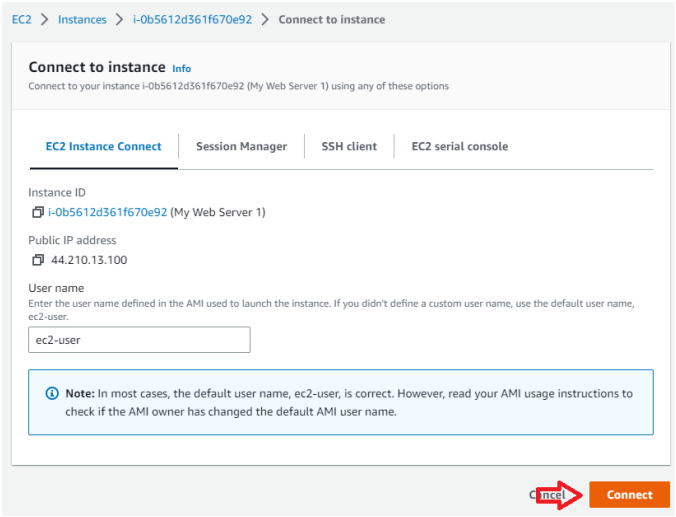
Once the Elastic IP address is allocated, go to Action and select Associate Elastic IP address in the drop-down menu.



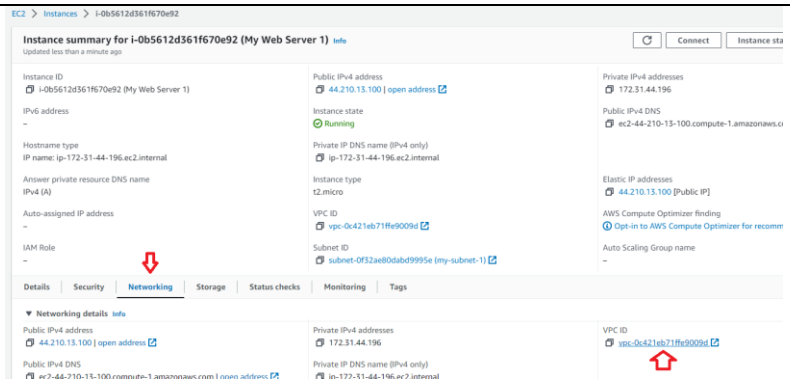
Choose My Web Server 1 as the instance and Click **Associate**.



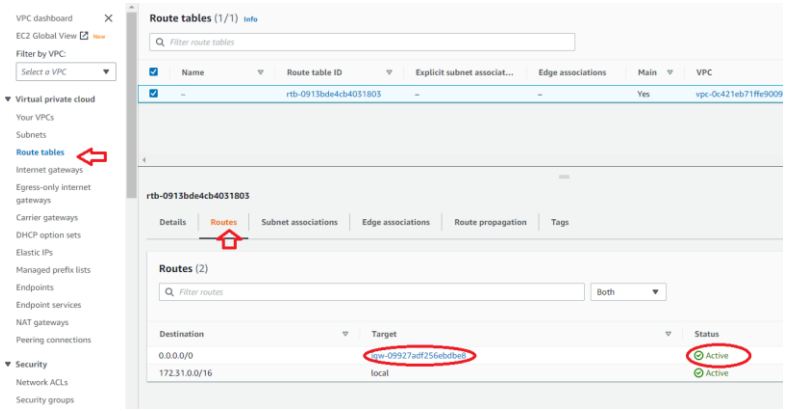
Trouble shooting 2: If cannot connect to EC2 instance.



Go to instance details, check Networking and click the link under VPC ID



Choose Route Tables, check Routes if the internet gateway is active.



Trouble shooting 3: If connecting to the EC2 instance is ok but web server is not running, rerun the following command lines to start the web server.

```
sudo yum update -y
sudo yum install httpd -y
sudo systemctl start httpd
sudo systemctl enable httpd

echo '<html><h1>Hello From Web Server 1!</h1></html>' > a.html
sudo cp a.html /var/www/html/index.html
rm a.html

sudo chown apache:apache /var/www/html/index.html
sudo chmod 644 /var/www/html/index.html

sudo systemctl status httpd
```

Check the URL in a web browser
http://<public_ip_address>

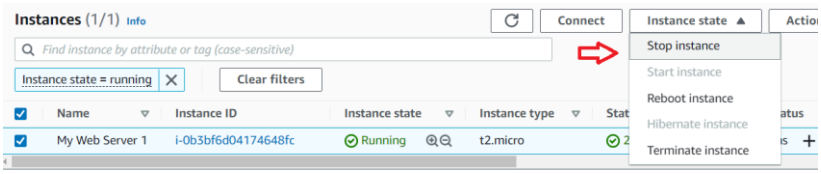
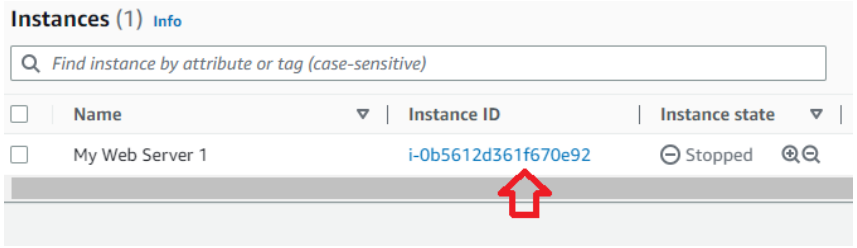
Open a web browser and check the URL

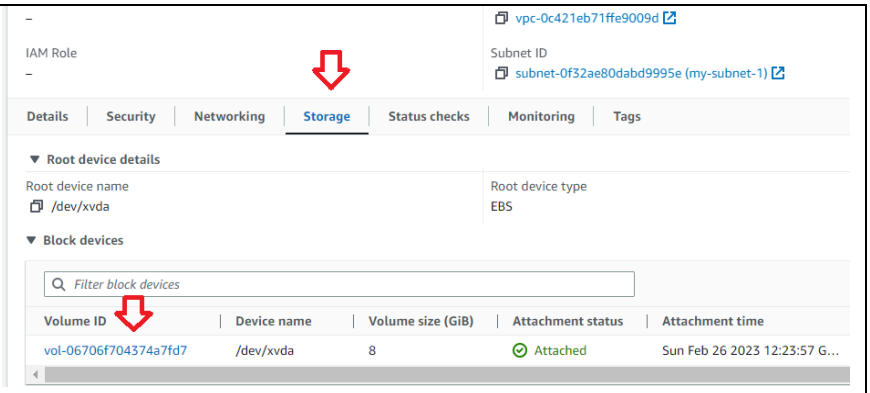
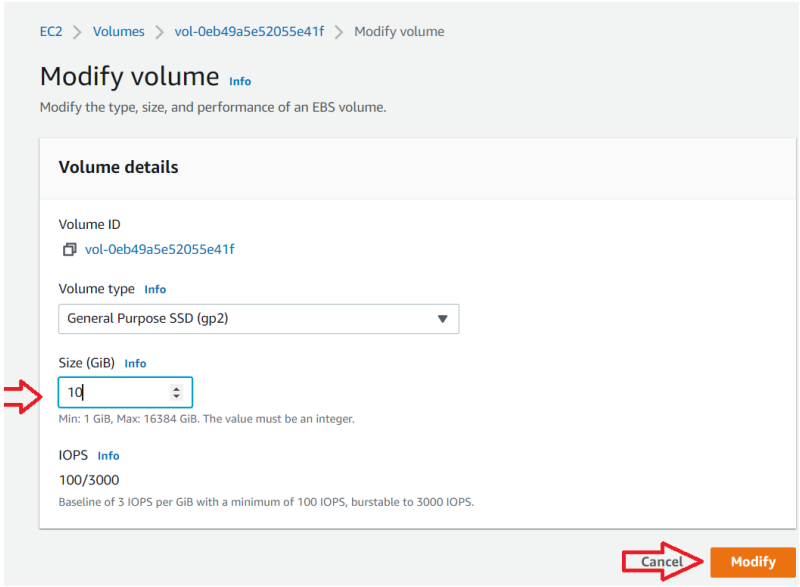
http://<public_ip_address>

Add the screenshot to Result 1 in the lab report.

Task 2: Configure the web server instance

You can follow the steps:

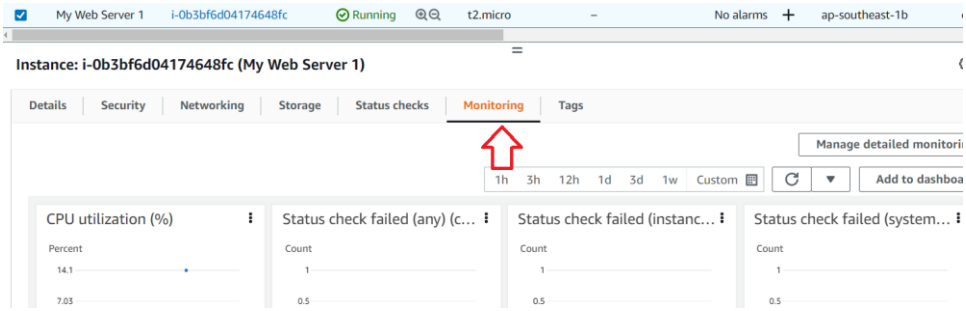
SN	Steps	Instructions
1)	Stop the instance	Select My Web Server 1 instance, choose **Instance State -> Stop instance**. 
2)	Change instance type	Click **Actions**, select **Instance settings** ->**Change instance type**, then configure: • **Instance Type:** *t2.small* • Choose **Apply**
3)	Resize the EBS Volume	In the instance details panels, choose the **Storage** tab, click the Volume ID. 

		 Select the volume, click **Actions**, **Modify volume**. 
5)	Start the instance	Select the **My Web Server 1** instance. In the **Instance state** menu, select **Start instance**.
6)	Check the instance details	Note that the instance type and storage size are updated accordingly and highlight the updates. **Add the screenshot to Result 2 in the lab report.**

Task 3: Monitor the performance

You can follow the steps:

Choose the **Monitoring** tab.

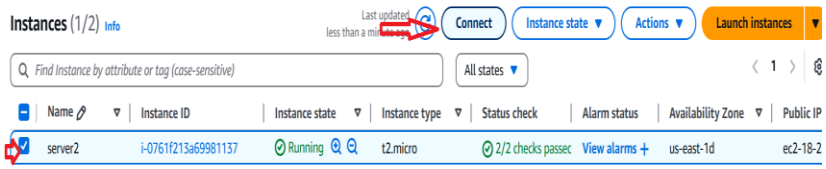


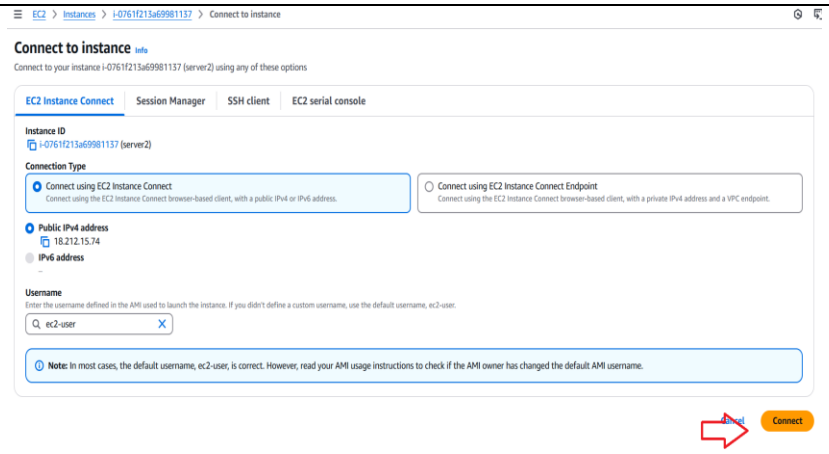
This tab displays Amazon CloudWatch metrics for the web server 1 instance. You can choose the three dots icon in any graph and select **Enlarge** to see an expanded view of the chosen metric. Basic (five-minute) monitoring is enabled by default. You can also enable detailed (one-minute) monitoring.

Add the screenshot to Result 3 in the lab report.

Task 4: Create a microservice using Docker container

You can repeat task 1 to create a Server 2 then follow the steps:

SN	Steps	Instructions
1)	Connect to the instance	Select the instance and click Connect. 

		 Click **Connect**

1)	Create a microservice python file named web_service.py	Run the vim command
		
		$vim web_service.py
		
		Press “i” and copy and paste the following content.
		
		`from flask import Flask`
		`app = Flask(__name__)`
		
		`@app.route('/')`
		`def home():`
		`  return 'Hello! This is from Web Server 2.'`
		
		`if __name__ == '__main__':`
		`  app.run(host='0.0.0.0', port=80)`
		
		Press “esc” and “:wq” to save it.
		
		Vim commands for reference:
		
		○ Press “i” to change to insert mode
		○ Press “esc”, then enter “:w” to save the file without exiting vim
		○ Press “esc”, then enter “:wq” to save the file and exit vim

2)	Create a Dockerfile named Dockerfile-web	Create a Dockerfile to specify how a microservice should be containerized. $vim Dockerfile-web Press “i” and copy and paste the following content.

		FROM python:3.8-slim COPY web_service.py /app/ WORKDIR /app RUN pip install Flask CMD ["python", "web_service.py"] Press “esc” and “:wq” to save it.

3	Install Docker in the EC2 instance	Install Docker by running the command lines
		
		$sudo yum update -y
		
		$ sudo yum install docker -y
		
		Start the docker service
		
		$sudo systemctl start docker
		
		Enable docker to start on boot
		
		$ sudo systemctl enable docker
		
		Stop the Apache server (if you have run the Apache as in the 1st instance)
		
		$ sudo systemctl stop httpd

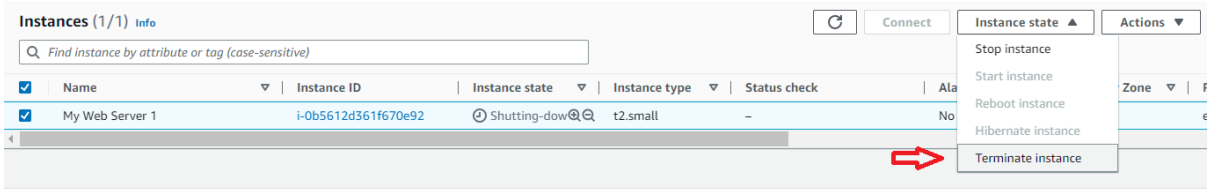
4)	Build and Run the Container	Build the container image $ sudo docker build -t web-service -f Dockerfile-web . Check the images created $sudo docker images Run the container $sudo docker run -d -p 80:80 web-service Check the process running $sudo docker ps

Open a web browser and type http://<public_ip_address of the EC2 instance> to view the content.

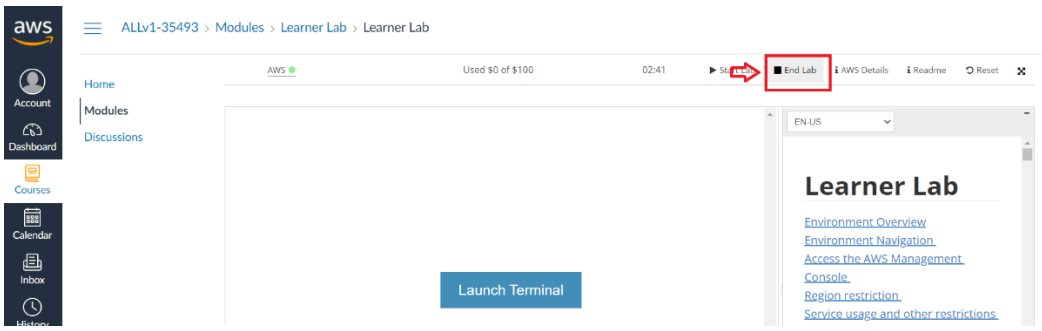
Add the screenshot to Result 4 in the lab report.

Task 5: Terminate the instance

Select the My Web Server 1 and Server 2 instances in the EC2 Dashboard, and choose **Terminate instance** in the box of Instance State.



Log out from AWS console. Click End Lab from AWSAcademy Canvas.



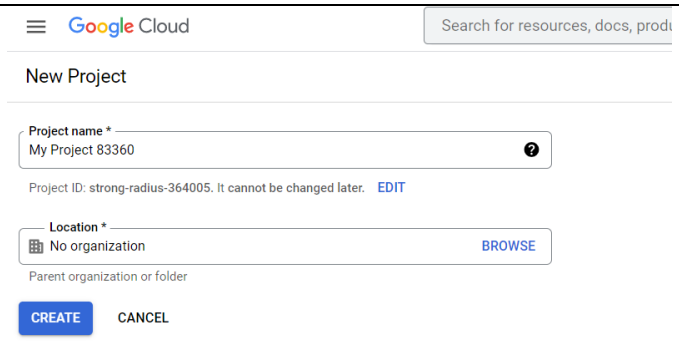
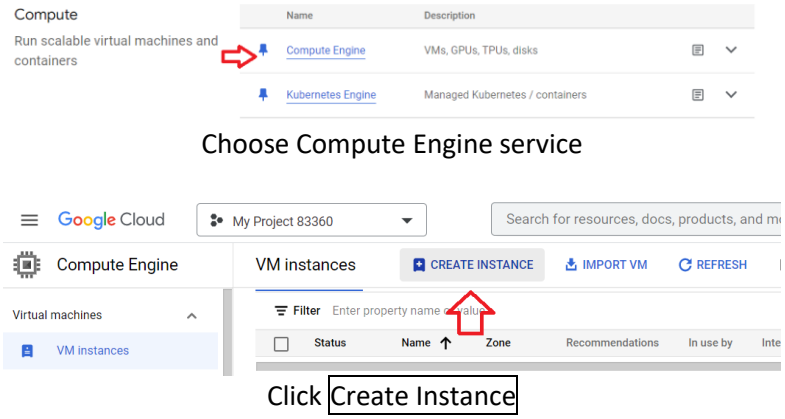
Add one or two sentences to summarize what you have learned to the lab report.

Part 2: (Optional) Build a web server in Google Cloud

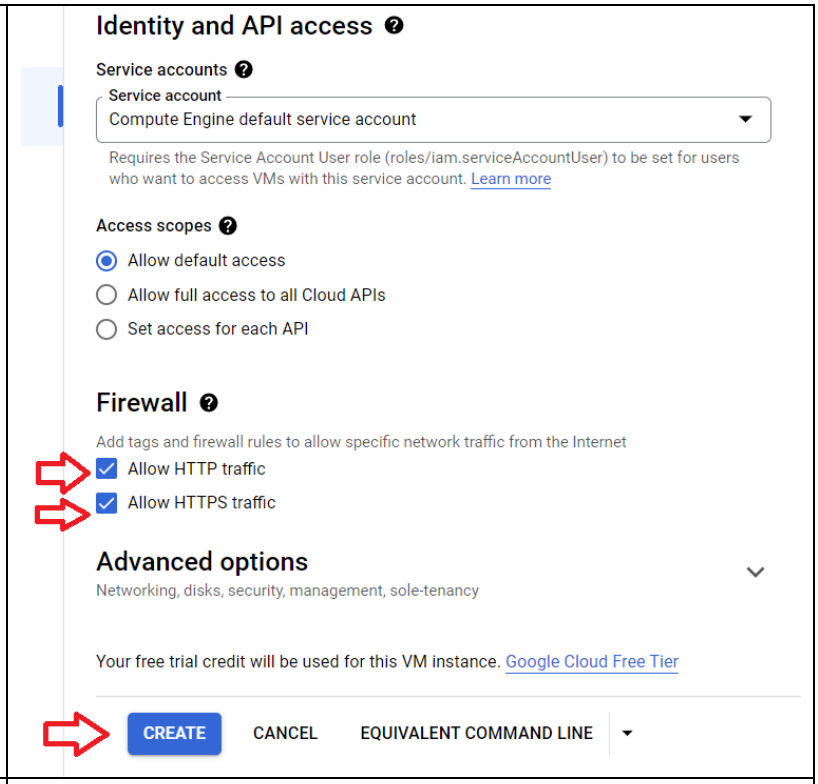
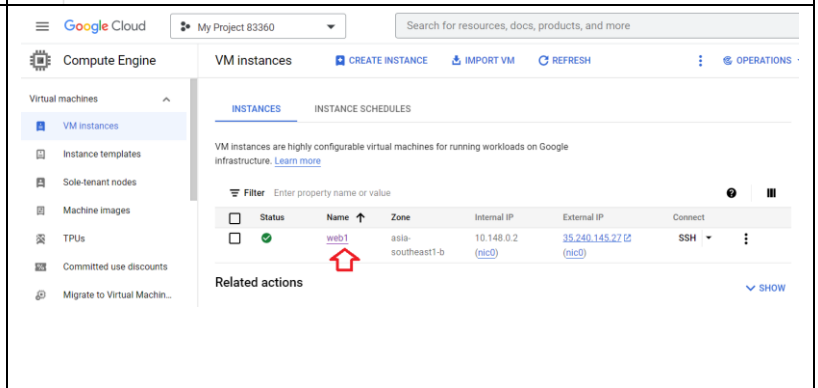
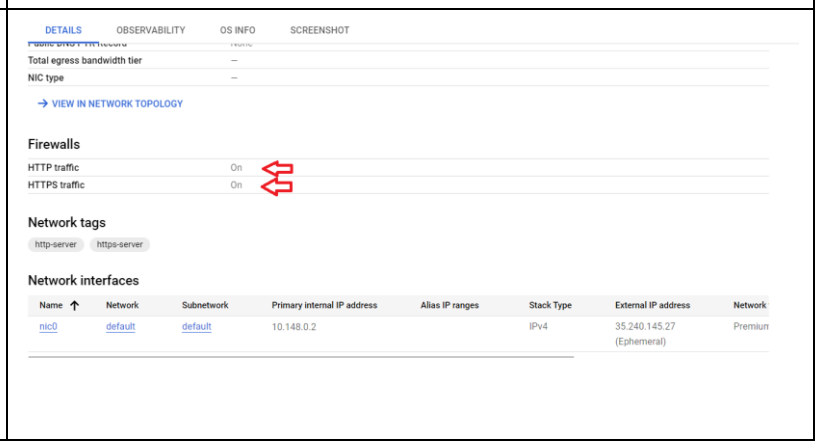
A Google Compute Engine instance will be launched to run a simple web server.

Task 1: Launch a web server using a Google Compute Engine instance

Go to **Google Cloud Console**, choose **Compute Engine**.

SN	Steps	Instructions
1)	Name and create a project	
2)	Choose **Compute Engine** and click **Create Instance**	 Choose Compute Engine service Click Create Instance

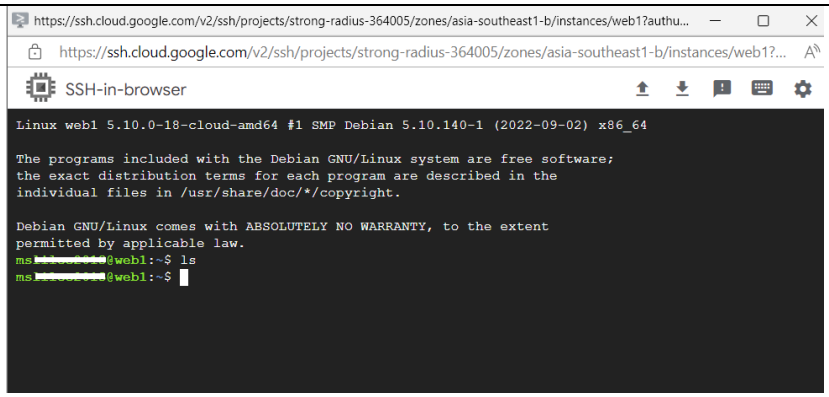
3)	Name the instance, specify the instance type, region, etc.	Name * web1 Labels + ADD LABELS Region * asia-southeast1 (Singapore) — Region is permanent Zone * asia-southeast1-b — Zone is permanent **Machine configuration** Machine family GENERAL-PURPOSE COMPUTE-OPTIMIZED MEMORY-OPTIMIZED GPU Machine types for common workloads, optimized for cost and flexibility Series E2 CPU platform selection based on availability Machine type e2-micro (2 vCPU, 1 GB memory) vCPU 0.25-2 vCPU (1 shared core) Memory 1 GB
4)	Configure display, access, storage	**Display device** Enable to use screen capturing and recording tools. ☑ Enable display device **Confidential VM service** Confidential Computing is disabled on this VM instance ENABLE **Container** Deploy a container image to this VM instance DEPLOY CONTAINER **Boot disk** Name: web1 Type: New balanced persistent disk Size: 10 GB License type: Free Image: Debian GNU/Linux 11 (bullseye) CHANGE

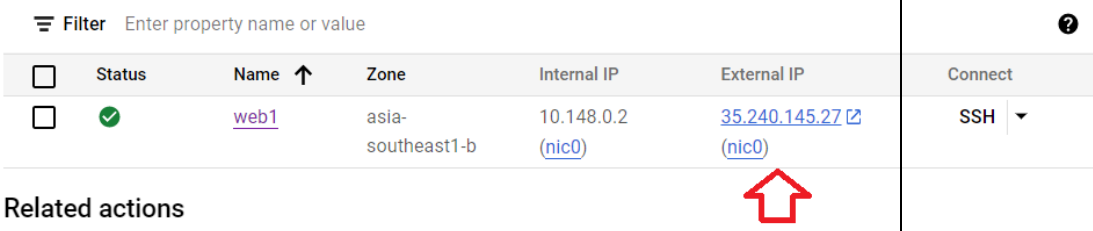
5)	Configure the identity, firewall and click CREATE	
6)	View the instance and check its status	
7)	View the instance specs	

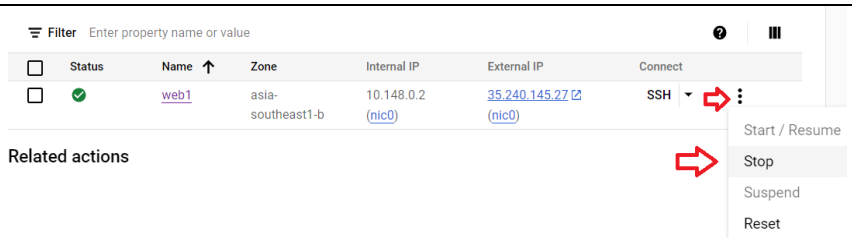
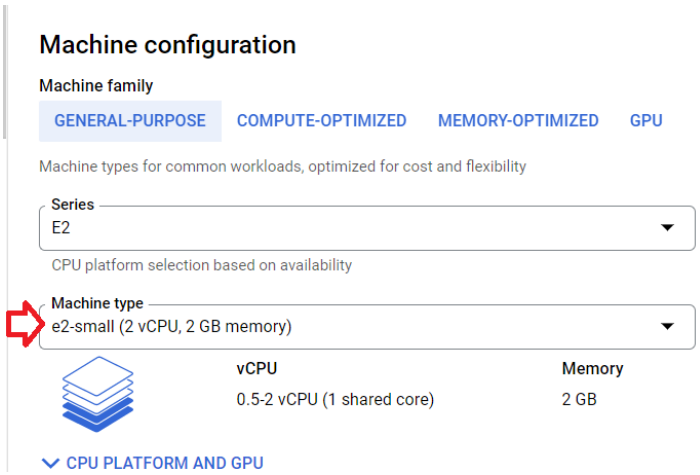
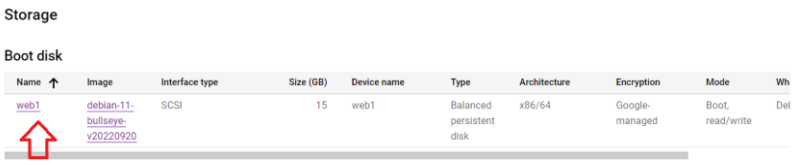
Task 2: Configure the web server instance

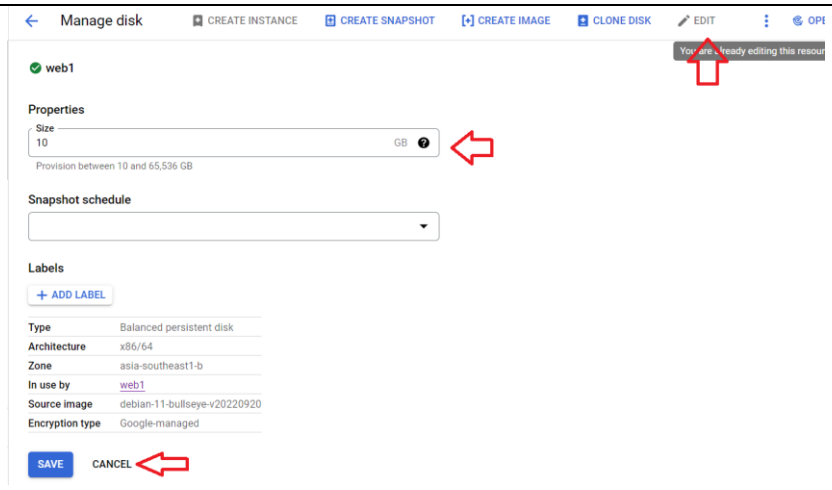
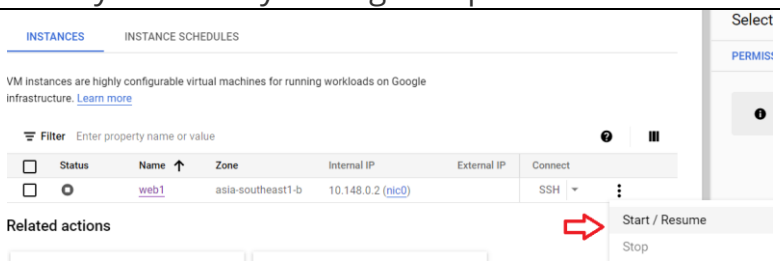
You can follow the steps:

SN	Steps	Instructions
1)	SSH into the instance	Click the SSH to connect to the VM instance

2)	Install and update the software	
		
		Update the software on the instance
		`$sudo apt-get update`
		`$sudo apt-get install apache2`

3)	Get the public IP Address of the instance and	 Edit the web page at /var/www/html/index.html with the content `<html><h1>Hello From Web Server!</h1></html>` Open a web browser and try the url http://<IP_address>

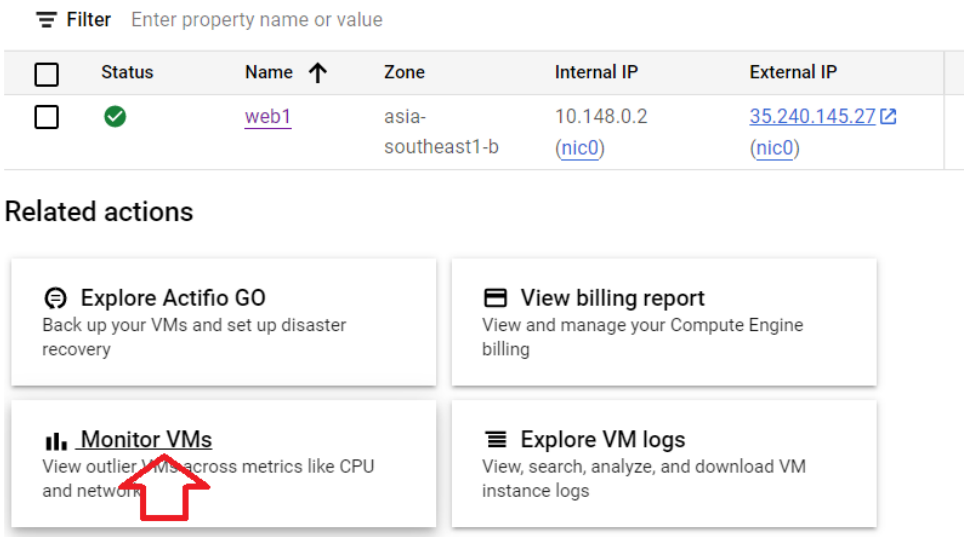
4)	Stop the instance	
5)	Change the type of the instance	Choose Edit to edit the configuration of the instance 
6)	Change the size of the disk and save it	In the left pane, select Disk, choose the target disk and click Edit.  Go to the boot disk to resize it

		 Modify the size by editing the specs
7)	Start the instance	 Check the instance details.

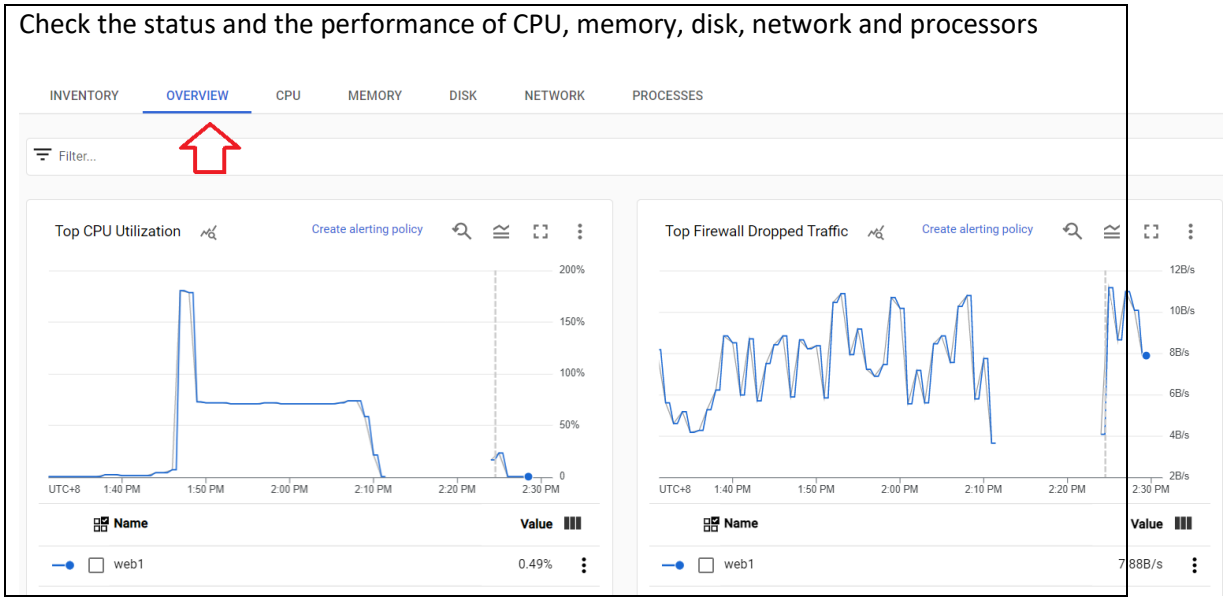
Task 3: Monitor the performance

You can follow the steps:

Choose the tab of **Monitoring VMs**.

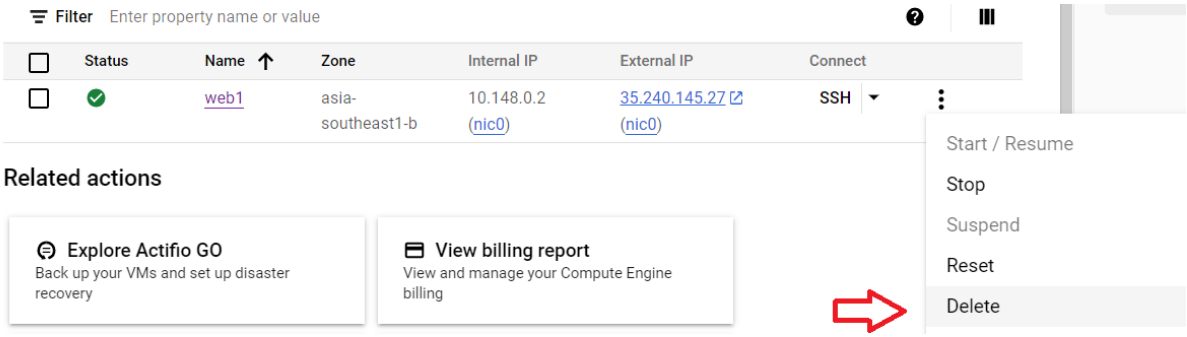


Check the status and the performance of CPU, memory, disk, network and processors



Task 4: Delete the instance

Select the Web 1 instance and choose Delete to delete the instance.

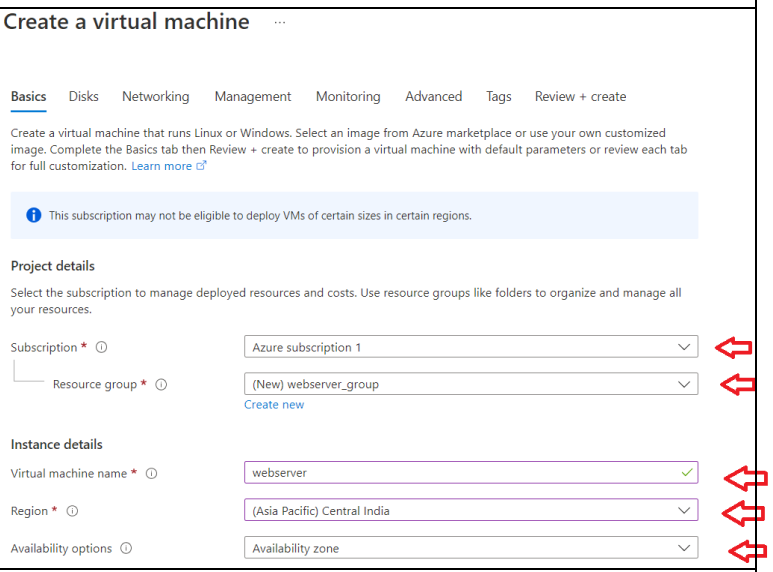
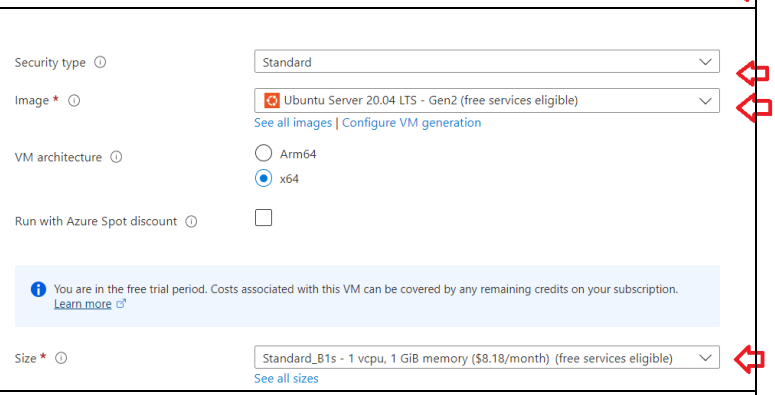
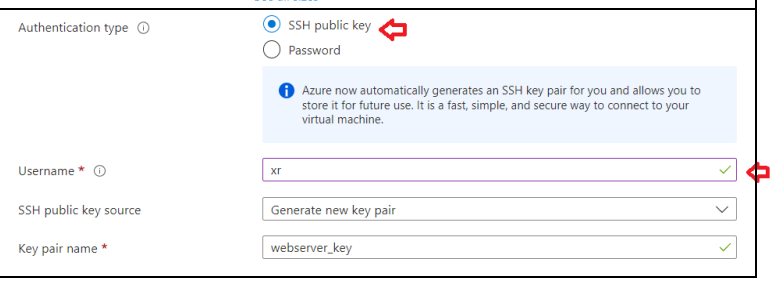


Part 3: (Optional) Build a web server in Azure Cloud

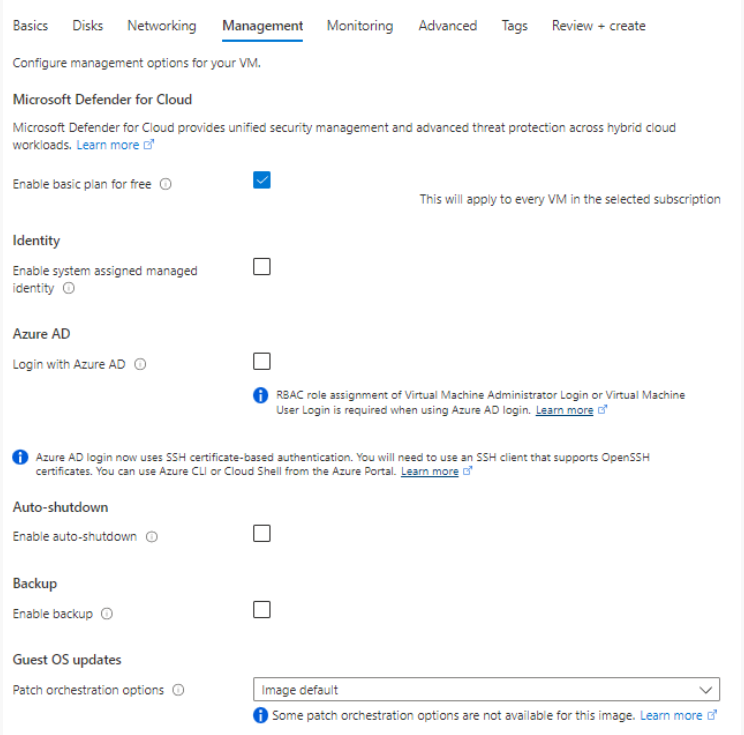
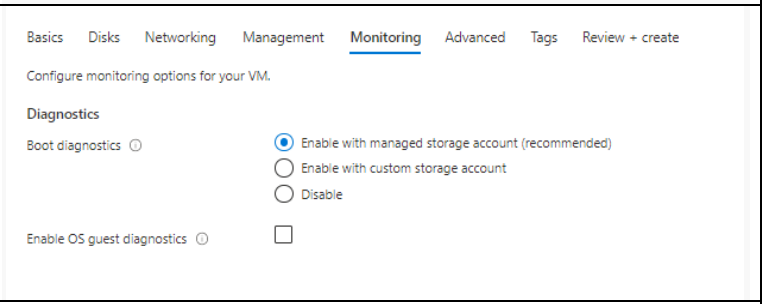
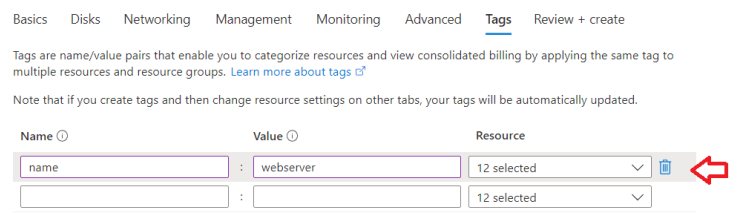
An Azure Virtual Machine instance will be launched to run a simple web server.

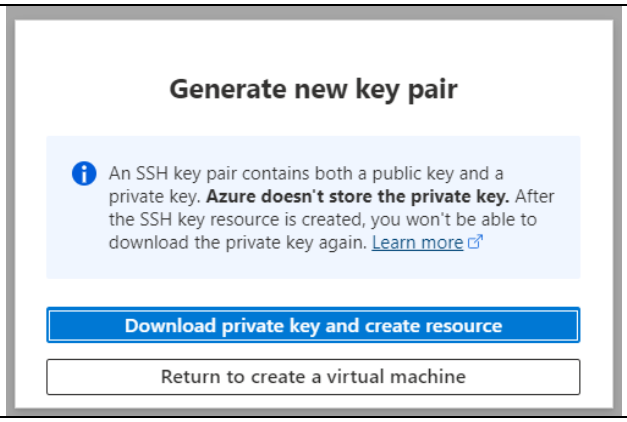
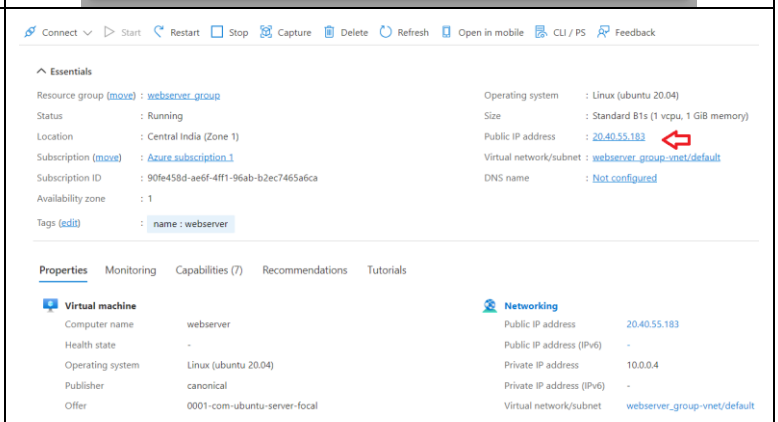
Task 1: Create a web server using an Azure Virtual Machine instance

Go to **Azure portal**, choose **Virtual Machine** to start

SN	Steps	Instructions
1)	Name the VM and select the region and zone	
2)	Choose the instance type	
3)	Configure the key generation	

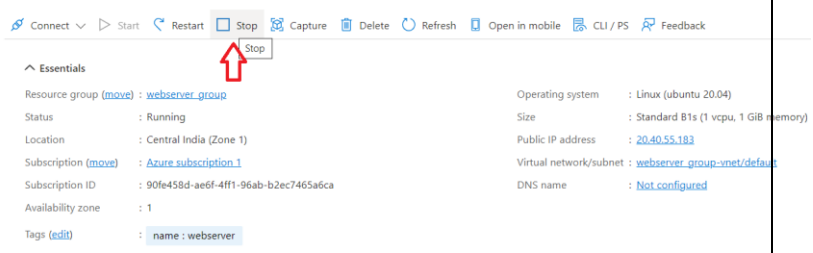
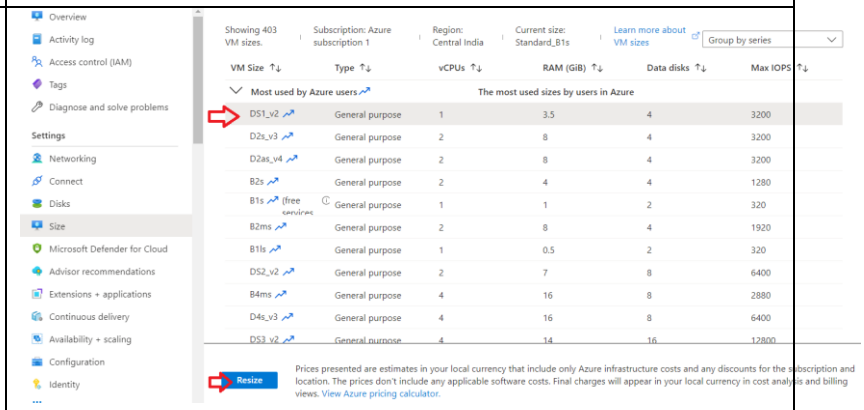
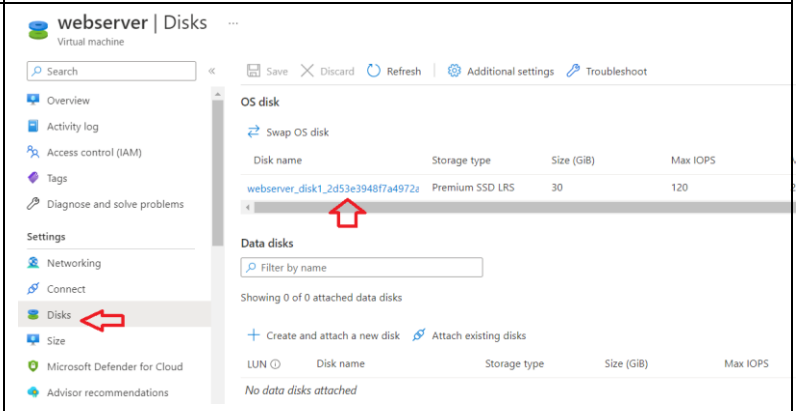
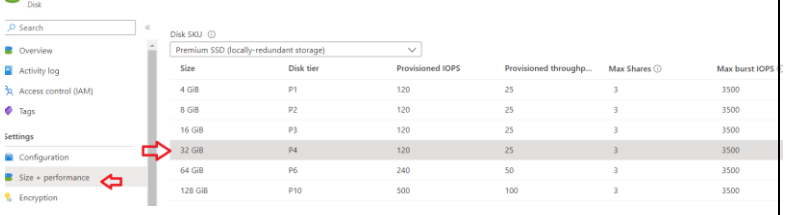
4)	Define the inbound port rules	**Inbound port rules** Select which virtual machine network ports are accessible from the public internet. You can specify more limited or granular network access on the Networking tab. Public inbound ports * ⓘ — ○ None / ◉ Allow selected ports Select inbound ports * — HTTP (80), HTTPS (443), SSH (22) ☑ HTTP (80) ☑ HTTPS (443) ☑ SSH (22)
5)	Configure the disk	**Disk options** OS disk size * ⓘ — Default size (30 GiB) OS disk type * ⓘ — Premium SSD (locally-redundant storage) Delete with VM ⓘ — ☑ Enable encryption at host ⓘ — ☐ ⓘ Encryption at host is not registered for the selected subscription. Learn more about enabling this feature Encryption type * — (Default) Encryption at-rest with a platform-managed key Enable Ultra Disk compatibility ⓘ — ☐ **Data disks for webserver** You can add and configure additional data disks for your virtual machine or attach existing disks. This VM also comes with a temporary disk. LUN | Name | Size (GiB) | Disk type | Host caching | Delete with VM ⓘ
6)	Configure the networking	**Network interface** When creating a virtual machine, a network interface will be created for you. Virtual network * ⓘ — (new) webserver_group-vnet Create new Subnet * ⓘ — (new) default (10.0.0.0/24) Public IP ⓘ — (new) webserver-ip Create new NIC network security group ⓘ — ○ None / ◉ Basic / ○ Advanced Public inbound ports * ⓘ — ○ None / ◉ Allow selected ports Select inbound ports * — HTTP (80), HTTPS (443), SSH (22) ⚠ **This will allow all IP addresses to access your virtual machine.** This is only recommended for testing. Use the Advanced controls in the Networking tab to create rules to limit inbound traffic to known IP addresses. Delete public IP and NIC when VM is deleted ⓘ — ☐ Enable accelerated networking ⓘ — ☐ The selected VM size does not support accelerated networking. **Load balancing** You can place this virtual machine in the backend pool of an existing Azure load balancing solution. Learn more Load balancing options ⓘ — ◉ None / ○ Azure load balancer (Supports all TCP/UDP network traffic, port-forwarding, and outbound flows.) / ○ Application gateway (Web traffic load balancer for HTTP/HTTPS with URL-based routing, SSL termination, session persistence, and web application firewall.)

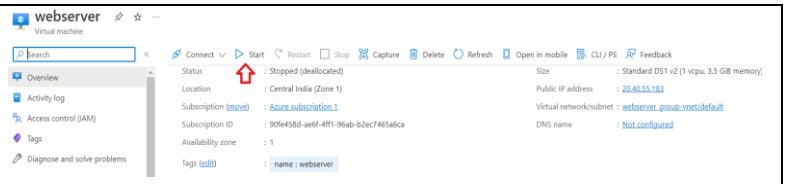
7)	Configure the VM management	
8)	Configure Monitoring	
9)	Create a tag	
10)	Review and Create	Review the specs and click Create to create the VM.

11)	Download the private key	
12)	Check the status	
13)	Update the web server file	SSH into the VM instance $ `ssh -i webserver_key.pem xr@20.40.55.183` Install apache web server $ `sudo apt install apache2` Modify the web page file $ `sudo vim /var/www/html/index.html` Enter the following into the index.html file `<html><h1>Hello From Web Server!</h1></html>`
14)	Get the public IP address	Open a web browser and check the URL http://<public_ip_address>

Task 2: Configure the web server instance

You can follow the steps:

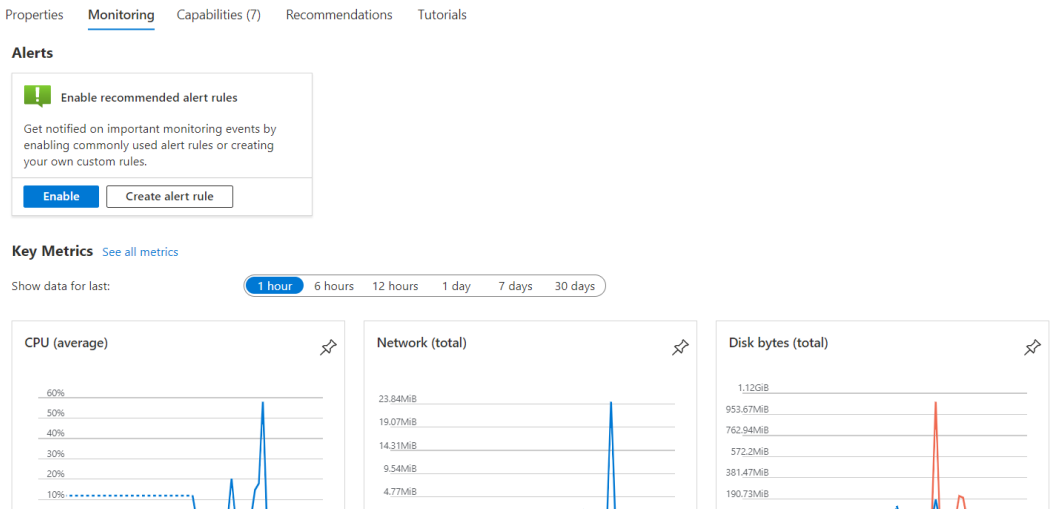
SN	Steps	Instructions
1)	Stop the instance	Select **Stop** to stop the instance. 
2)	Select the Size to specify the new instance type and select Resize	
3)	Select the disk, the disk name. Choose the Size + Performance, and select the size and click Resize at the bottom of the page.	 

4)	Restart the instance	

Task 3: Monitor the performance

You can follow the steps:

Choose the **Monitoring** tab.



This tab displays performance metrics for the web server instance. You can also enable the alert rules to generate alerts automatically when performance exceeds certain threshold.

Task 4: Terminate the instance

Choose **Stop** tab to stop the VM instance and then choose **Delete** to delete it.

Lab complete

Congratulations! You have completed the lab.

Please complete the lab report and submit it to x-site by the end of 10 Mar 2026 (Tuesday).

Thank you.